Universidad Nacional de Costa Rica
sede regional chorotega

EIF 201 Programación I

PROYECTO GRUPAL
BLACKJACK EN CONSOLA (C++)

Integrantes:

Nombre completo	Cédula
Yarath Valeria Rojas Jimenez	5-0463-0740
Franciny Baltodano Morales	5-0460-0010
Emilie Castrillo Espinoza	5-0466-0073
Yeikel Orias Chavarría	5-0444-0042
Hans Gamboa	2-0875-0104

Profesor: Francisco Coulon

Fecha de entrega de la Fase 1: 30 de octubre del 2026

repositorio Github:https://github.com/FranBaltodano/ProyectoProgracion-I

UNA
UNIVERSIDAD
NACIONAL
COSTA RICA

1.Introducción

El Blackjack es un juego de cartas en el que el objetivo principal es obtener una mano cuyo valor sea lo más cercano posible a 21, sin pasarse de ese número. Los jugadores compiten contra el crupier, siguiendo las reglas establecidas para repartir las cartas, solicitar cartas adicionales y determinar el resultado de cada ronda.

Este proyecto consiste en desarrollar una versión del juego Blackjack utilizando el lenguaje de programación C++ y los principios de la programación orientada a objetos. Para esto, se plantea un diseño basado en clases que permitan representar las cartas, el mazo, las manos, los jugadores, el crupier y el funcionamiento general del juego.

El sistema deberá utilizar un zapato compuesto por seis mazos de cartas, permitir barajar y repartir las cartas, calcular el valor de las manos y establecer los resultados de las partidas. Aparte , se documentarán las reglas del juego, el diseño de las clases, la distribución de las tareas y las pruebas realizadas durante el desarrollo.

El propósito de este proyecto es aplicar los conocimientos adquiridos en el curso mediante la construcción de un programa organizado, comprensible y funcional, que permita demostrar el uso de clases, atributos, métodos y relaciones entre objetos.

UNA
UNIVERSIDAD
NACIONAL
COSTA RICA

2.Definición del problema

Se requiere desarrollar un programa en C++ que simula una partida de Blackjack con varios jugadores y un crupier controlado por el sistema.

El programa debe administrar las cartas disponibles, repartirlas correctamente, calcular el valor de cada mano y permitir que los jugadores realicen las acciones establecidas por las reglas. Al finalizar cada ronda, el sistema debe comparar las manos de los jugadores con la mano del crupier y determinar si cada jugador gana, pierde o empata.

También se debe contemplar el manejo de apuestas, los pagos correspondientes y las situaciones especiales del juego, como el Blackjack natural, doblar la apuesta y dividir una pareja de cartas, de acuerdo con las reglas definidas para el proyecto.

2.1 Entradas del sistema

Las principales entradas del programa son:

- Las apuestas realizadas por los jugadores.
- Las cartas repartidas del zapato de seis mazos.
- Las acciones seleccionadas por el jugador, como pedir carta, plantarse, doblar o dividir, cuando corresponda.
- Las decisiones del sistema para ejecutar las acciones del crupier según las reglas establecidas.

El programa tiene que realizar las siguientes operaciones:

1. Crear un zapato con seis mazos de cartas.
2. Barajar las cartas antes de comenzar a jugar.

3. Repartir las cartas iniciales a los jugadores y al crupier.
4. Calcular el valor de cada mano, considerando el valor especial del as.
5. Validar las acciones permitidas para cada jugador.
6. Ejecutar el turno del crupier según las reglas del juego.
7. Comparar las manos y determinar el resultado de cada jugador.
8. Calcular los pagos y actualizar los saldos o resultados de las apuestas.

2.2 Salidas del sistema

- Las cartas recibidas por cada jugador y por el crupier, según corresponda.
- El valor total de las manos.Las acciones realizadas durante la ronda.
- El resultado final de cada jugador.
- El monto ganado, perdido o devuelto por cada apuesta.
- El saldo actualizado del jugador, si el sistema contempla el manejo de saldo.

2.3 Alcance

1. Incluido: La lógica completa de las reglas, apuestas y pagos descrita en la sección 3, un zapato de 6 mazos barajado una única vez por partida y un jugador humano contra el crupier.
2. Fuera de alcance: Interfaz gráfica, conteo de cartas automatizado, persistencia de resultados entre ejecuciones, juego en red, varios jugadores simultáneos y el seguro

.

3. Reglas del juego y ejemplos de uso

Ejemplo 1. VICTORIA DEL JUGADOR



Jugador: 19 puntos (10 + 9)

Crupier: 18 puntos (10 + 6 + 2)

El jugador obtiene un total de 19 puntos, mientras que el crupier obtiene 18 puntos.

Resultado: El jugador gana porque tiene más puntos que el crupier, sin superar los 21 puntos.

- Apuesta inicial: ₡1.000
- Ganancia neta: ₡1.000
- Pago total recibido: ₡2.000

Ejemplo 2: DERROTA DEL JUGADOR



Jugador: 18 puntos (10 + 8)

Crupier: 20 puntos (10 + 7 + 3)

El jugador obtiene un total de 18 puntos, mientras que el crupier obtiene 20 puntos.

Resultado: El jugador pierde porque el crupier tiene un valor superior en su mano.

- Apuesta inicial: ₡1.000
- Pérdida: ₡1.000
- Pago recibido: ₡0

Ejemplo 3: EMPATE



Jugador: 19 puntos (10 + 9)

Crupier: 19 puntos (10 + 6 + 3)

El jugador y el crupier obtienen un total de 19 puntos cada uno.

Resultado: Se produce un empate porque ambos tienen el mismo valor en sus manos. El jugador recibe de vuelta el dinero que había apostado.

- Apuesta inicial: ₡1.000
- Ganancia neta: ₡0
- Monto devuelto: ₡1.000

Ejemplo 4: BLACKJACK NATURAL



Jugador: 21 puntos (As(11) + rey(10))

Crupier: 17 puntos (10 + 7)

El jugador obtiene un As (A) y un Rey (K) en sus dos primeras cartas, alcanzando un total de 21 puntos. Por su parte, el crupier obtiene 17 puntos.

Resultado: El jugador consigue un Blackjack natural. Recibe un pago de 3:2 sobre su apuesta inicial, además de la devolución de dicha apuesta.

- Apuesta inicial: ₡1.000
- Ganancia neta: ₡1.500
- Devolución de la apuesta: ₡1.000
- Pago total recibido: ₡2.500

Ejemplo 5: DOBLAR



Jugador: 21 puntos (5 + 6 + 10)

Crupier: 22 puntos (10 + 7 + 5)

El jugador comienza con una apuesta de ₡1.000 y decide utilizar la opción de doblar. Por lo tanto, su apuesta aumenta a ₡2.000 y recibe una sola carta adicional. Al finalizar, el jugador obtiene 21 puntos, mientras que el crupier supera los 21 puntos y pierde por pasarse.

Resultado: El jugador gana la ronda utilizando la apuesta duplicada.

- Apuesta inicial: ₡1.000
- Apuesta después de doblar: ₡2.000
- Ganancia neta: ₡2.000
- Pago total recibido: ₡4.000

4 Resumen de las reglas

4.1 Valor de las cartas

- Las cartas del 2 al 10 conservan su valor numérico.
- La jota (J), la reina (Q) y el rey (K) valen 10 puntos.
- El as (A) puede valer 1 u 11 puntos, dependiendo de cuál valor beneficie más a la mano sin superar 21.

4.2 Inicio de la partida

- La partida comienza con un zapato compuesto por 6 mazos de cartas.
- Cada mazo contiene 52 cartas, para un total de 312 cartas.
- Las cartas se barajan antes de comenzar a jugar.
- Se reparten las cartas iniciales a los jugadores y al crupier.

4.3 Acciones del jugador

Durante su turno, el jugador puede realizar las siguientes acciones:

- **Pedir**: recibir una carta adicional.
- **Plantarse**: finalizar su turno sin recibir más cartas.
- **Doblar**: duplicar la apuesta y recibir una sola carta adicional.
- **Dividir:** separar dos cartas iniciales del mismo valor en dos manos diferentes, con una segunda apuesta igual a la inicial. Solo se permite una vez (máximo 2 manos) y cada mano se juega y se resuelve de forma independiente.
- **Apuestas:** el jugador inicia con un saldo de 1000 y a**puesta entre 10 y 500**. Doblar y dividir solo se permiten si el saldo alcanza para cubrir la apuesta adicional.

4.4 Reglas del crupier

El crupier es controlado por el sistema y debe cumplir las siguientes reglas:

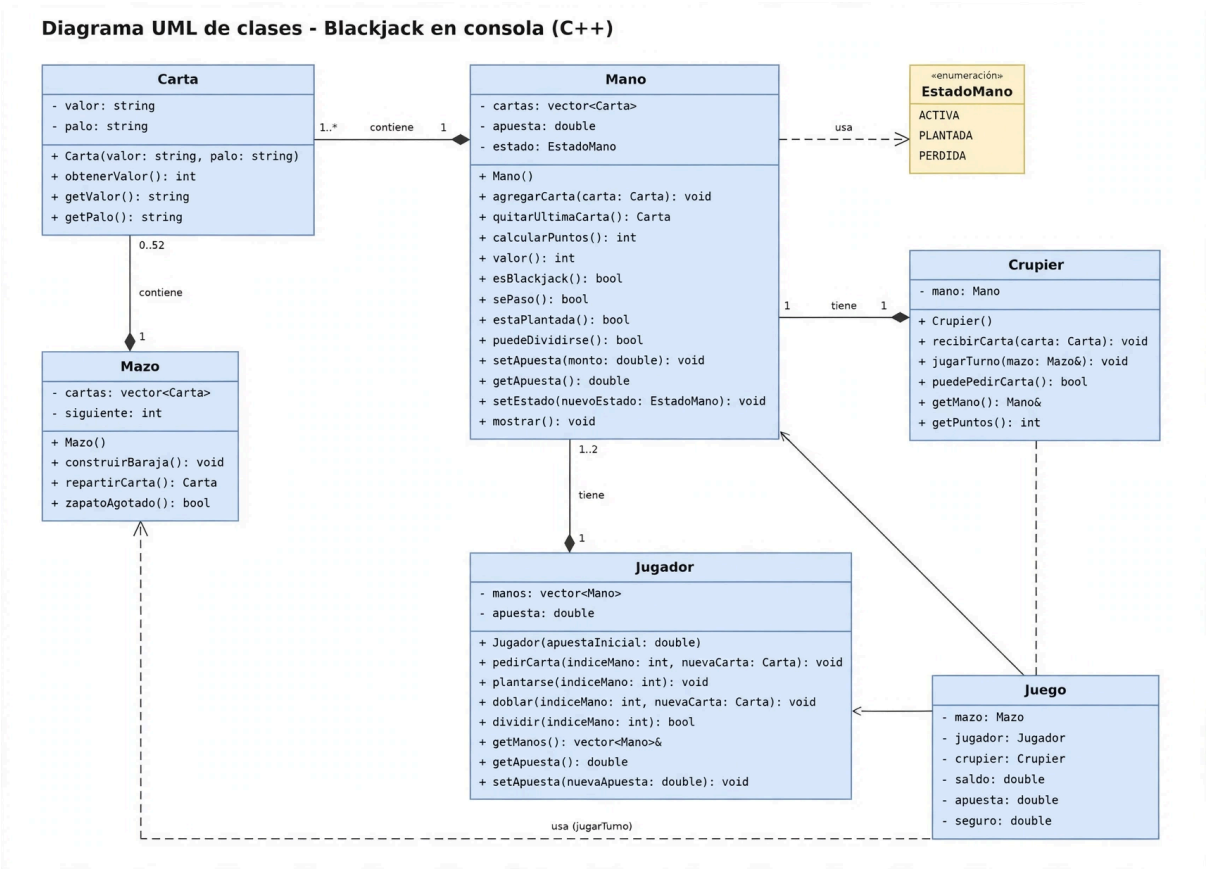
- Si tiene menos de 17 puntos, debe pedir otra carta.
- Si tiene 17 puntos o más, debe plantarse.
- Debe continuar solicitando cartas mientras su mano tenga menos de 17 puntos.
- Si supera los 21 puntos, pierde la ronda por pasarse
- Puede pedir solo 17 cartas más.

4.5 Resultados y pagos

Resultado	Condición	Pago
Victoria normal	El jugador supera al crupier	Recibe el doble de su

	sin pasarse de 21.	apuesta (incluye devolución de apuesta inicial).
Derrota	El crupier tiene una mano mayor o el jugador se pasa de 21.	Pierde el dinero apostado.
Empate	El jugador y el crupier tienen el mismo puntaje.	Recibe de vuelta el monto de su apuesta (sin ganancias adicionales).
Blackjack natural	El jugador obtiene un As y una carta de valor 10 en sus dos primeras cartas.	Pago de 3:2 sobre su apuesta, más devolución de apuesta inicial. Apuesta ₡1000 se le devuelven ₡1500 más haciendo ₡2500.
Doblar	El jugador duplica su apuesta y recibe una sola carta adicional.	Si gana, el pago se calcula usando la apuesta duplicada.
Dividir	Separa dos cartas iniciales iguales en dos manos y hace una segunda apuesta.	Cada mano se juega y resuelve de forma independiente.

5 Diagrama UML



Un Mazo (zapato de 6 barajas) contiene las Cartas. Cada Mano agrupa una o más Cartas y tiene un EstadoMano (ACTIVA, PLANTADA o PERDIDA). Un Jugador tiene una o dos Manos (dos tras dividir) y el Crupier tiene una Mano; el Crupier recibe el Mazo en jugarTurno(). El flujo de la partida (apuesta, reparto, turnos y pagos) lo controla el programa principal.

Clase	Atributos principales	Métodos principales	Responsabilidad
Crupier	Mano mano	recibirCarta(Carta), jugarTurno(Mazo), getMano(), getPuntos(), puedePedirCarta()	juega como crupier, pide cartas hasta llegar a 17
Jugador	vector<Mano> manos, double apuesta	Jugador(double), pedirCarta(int, Carta), plantarse(int), doblar(int,	Administra las manos del jugador y sus apuestas, incluida la

Clase	Atributos principales	Métodos principales	Responsabilidad
		Carta), dividir(int), getManos(), getApuesta(), setApuesta(double) getManos()	division en maximo 2 manos.
Mazo	vector<Carta> cartas, double apuesta, EstadoMano estado	construirBaraja(), repartirCarta(), zapatoAgotado()	Arma y baraja los 6 mazos una sola vez y reparte cartas hasta agotarse.
Mano	vector<Carta> cartas, double apuesta, EstadoMano estado	agregarCarta(Carta), quitarUltimaCarta(), calcularPuntos(), valor(), esBlackjack(), sePaso(), estaPlantada(), puedeDividirse(), setApuesta(double), getApuesta(), setEstado(EstadoMano), mostrar()	Calcula el valor de la mano ajustando el As a 1 u 11, detecta blackjack, si se paso de 21 y si se puede dividir.
Carta	string valor, string palo ını sıdnıeuıe	Carta(string, string), obtenerValor(), getValor(), getPalo()	Representa una carta. obtenerValor() devuelve 11 si es As, 10 si es J, Q o K, y el numero si es otra carta.
Juego	Mazo mazo, Jugador jugador, Crupier crupier, double saldo, double apuesta, double seguro	Juego(), jugar(), pedirApuesta(), ofrecerSeguro(Carta), jugarMano(int), jugarTurnoCrupier(), resolverResultados(), resolverBlackjackInicial (bool, bool, bool)	Controla toda la partida: pide la apuesta, reparte, ofrece seguro, controla los turnos y resuelve victoria, derrota y empate.

UNA
UNIVERSIDAD NACIONAL
COSTA RICA

Plan de acción del proyecto

Fase 1-Hasta 30 de octubre

Tarea	Responsable	Fecha	Depende de
Reunión de inicio y división de roles	Todo el equipo	24 de octubre	Nada
Diseño de clases y UML	Integrante 1: Franciny	25 de octubre	Reunión de inicio
Redacción: introduccion y definicion del problema	Integrante 2: Valeria	26 de octubre	Reunión de inicio
Redacción: reglas del juego y ejemplos enumerados	Integrante 3: Emilie	27 de octubre	Reunión de inicio
Implementación inicial (carta, mazo, mano: armar zapato, barajar, repartir, valor de mano)	Integrante 4: Yeikel	28 de octubre	Diseño de clases
Capturas del avance del código	Integrante 5: Hans Gamboa	29 octubre	Implementación inicial
Integración del documento formal en pdf y revisión	Todo el equipo	29 de octubre	Todo lo anterior

Tarea	Responsable	Fecha	Depende de
Ensayo	Todo el equipo	29 de octubre	Documento integrado
Entrega en moodle y exposición	Todo el equipo	30 de octubre	Ensayo

Fase 2 - Hasta 6 de noviembre

Tarea	Responsable	Fecha	Depende de
Crear repositorio en GitHub, estructura de carpetas	Integrante 1: Franciny	2 de noviembre	-
Implementar Jugador, Crupier y Juego (apuestas, turnos, pagos)	Integrante 2: Yeikel, integrante 3: Hans.	2 de noviembre	Clase base fase 1
Implementar seguro, dividir y doblar	Integrante 4: Emilie	3 de noviembre	Jugador, Crupier, Juego
Validación de entradas y pruebas de todas las reglas	Integrante 5: Valeria	4 de noviembre	Lógica completa
Documentación técnica: cómo jugar y guías de compilación (cl y Visual Studio)	Integrante 2: Yeikel	4 de noviembre	Juego funcionando
Commits regulares de cada integrante	Todo el equipo	Continuo desde el 1 de noviembre	Repositorio creado

con mensajes descriptivos			
Revisión final, captura del repositorio y documento formal actualizado	Todo el equipo	5 de noviembre	Todo lo anterior
Ensayo de la exposición de Fase 2	Todo el equipo	5 de noviembre	Revisión final
Entrega en Moodle y exposición	Todo el equipo	6 de noviembre	Ensayo

Minutas de reuniones

Minuta 1

Campo	Detalle
Fecha	26/10/2026
Modalidad	Presencial
Asistentes	Valeria, Franciny, Emilie, Yeikel y Hans.
Temas tratados	Reparto de roles y definición del diseño de clases.
Acuerdos	Cada integrante entrega su parte asignada antes del jueves 29 de octubre.
Responsables y fecha de seguimiento	Cada integrante va añadiendo su parte asignada al documento.

Minuta 2

Campo	Detalle
Fecha	3/11/2026
Modalidad	Presencial
Asistentes	Todos los integrantes del grupo
Temas tratados	Revisión del código y del documento formal.
Acuerdos	Ensayo para la exposición final del proyecto
Responsables y fecha de seguimiento	Cada integrante revisa el documento y el código.

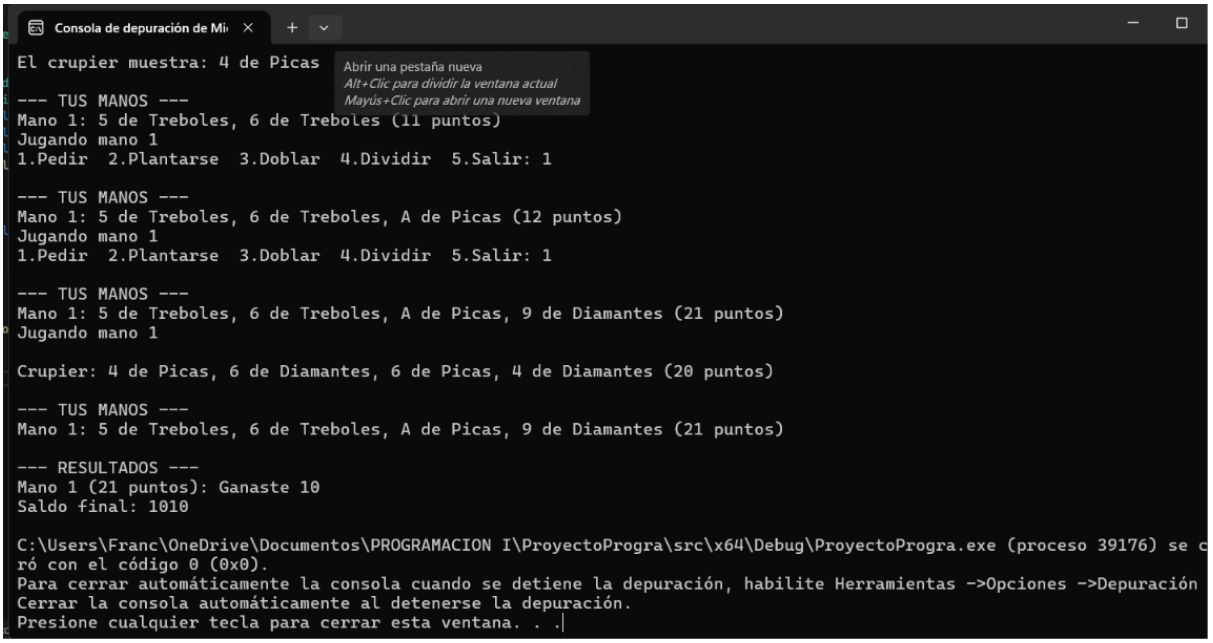
Avance de la implementación

Funcionalidades ya operativas

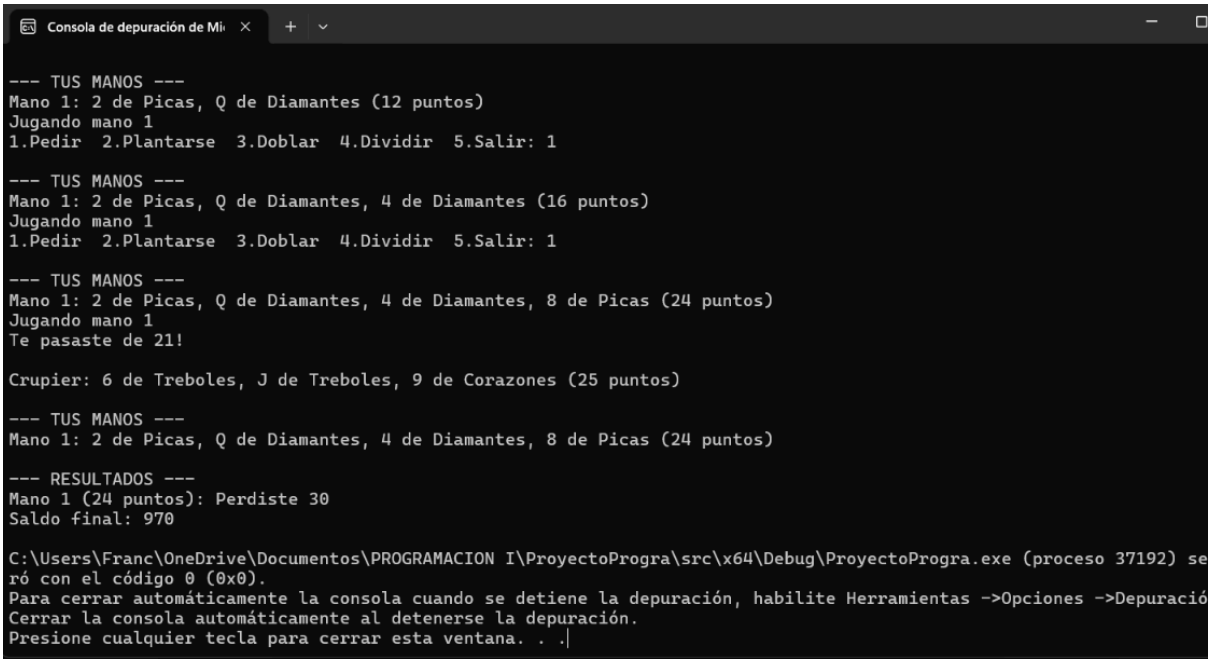
1. Construcción del zapato: construirBaraja() arma las 312 cartas (6 mazos × 52) y las baraja una única vez con std::shuffle.
2. Reparto: repartirCarta() entrega la siguiente carta del zapato.
3. Cálculo del valor de una mano: calcularPuntos() suma las cartas usando cada As este ofrece un seguro por si el crupier muestra As no perder toda la apuesta como 11 y lo baja a 1 (de uno en uno) mientras la mano se pase de 21 y queden ases por ajustar.
4. Flujo de una ronda: apuesta validada (10 a 500) contra un saldo inicial de 1000, reparto, turno del jugador (pedir, plantarse, doblar y dividir), turno del crupier (pide hasta 17 o más) y pago según la tabla de reglas (victoria 1:1, empate devuelve la apuesta, blackjack natural 3:2).
5. Validación de entradas: leerEnteroValido() rechaza datos inválidos y vuelve a preguntar sin interrumpir el programa; doblar y dividir validan que alcance el saldo.

Captura de una ejecución real:

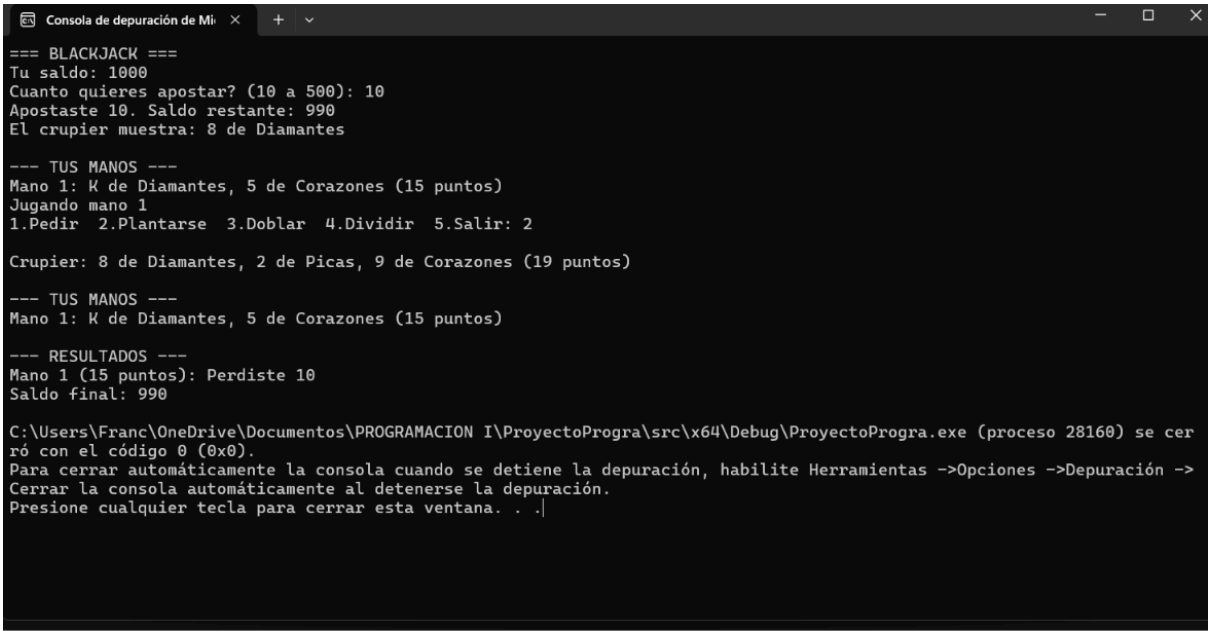
a) **Ganar:**



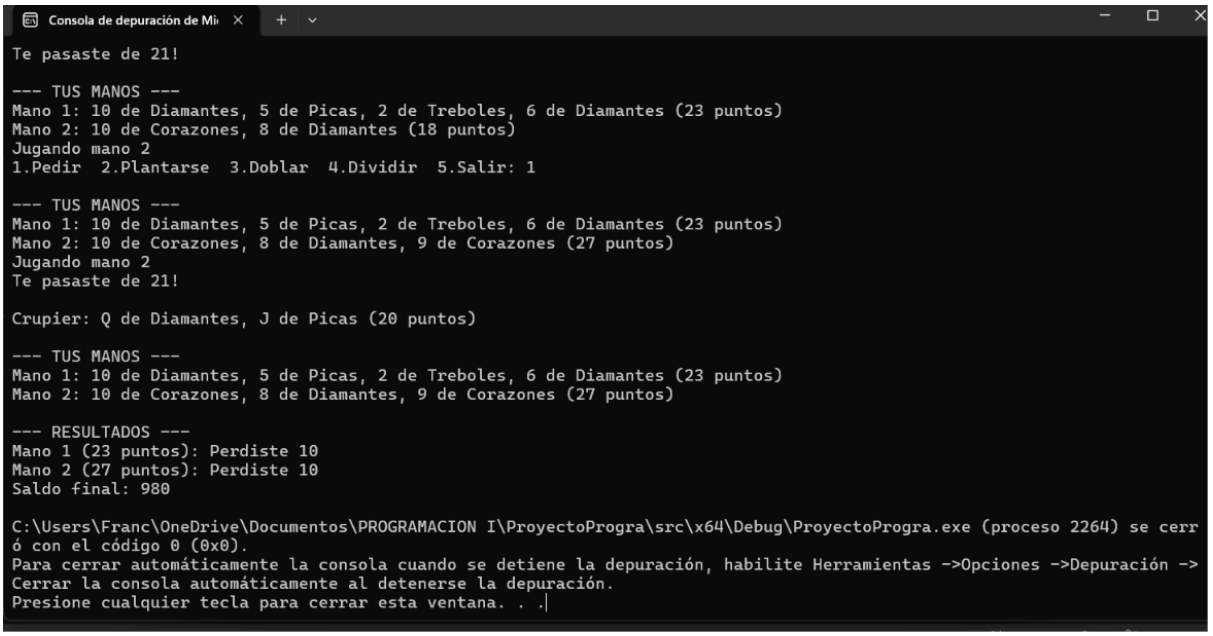
b)Perder



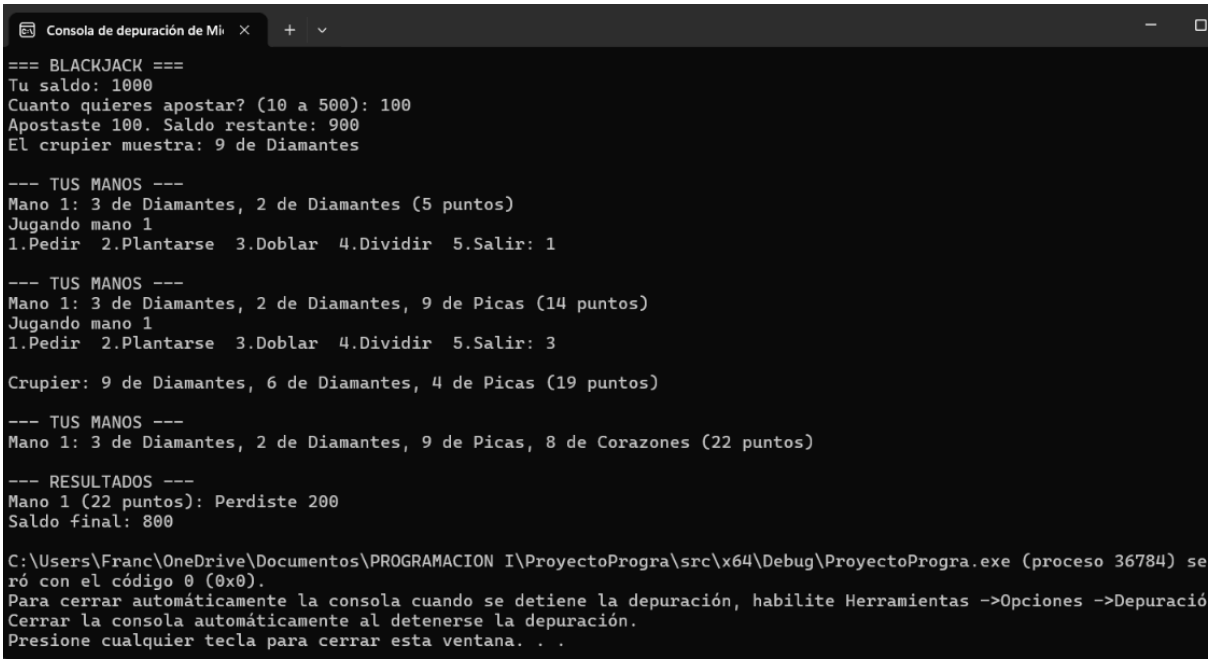
c)Plantarse



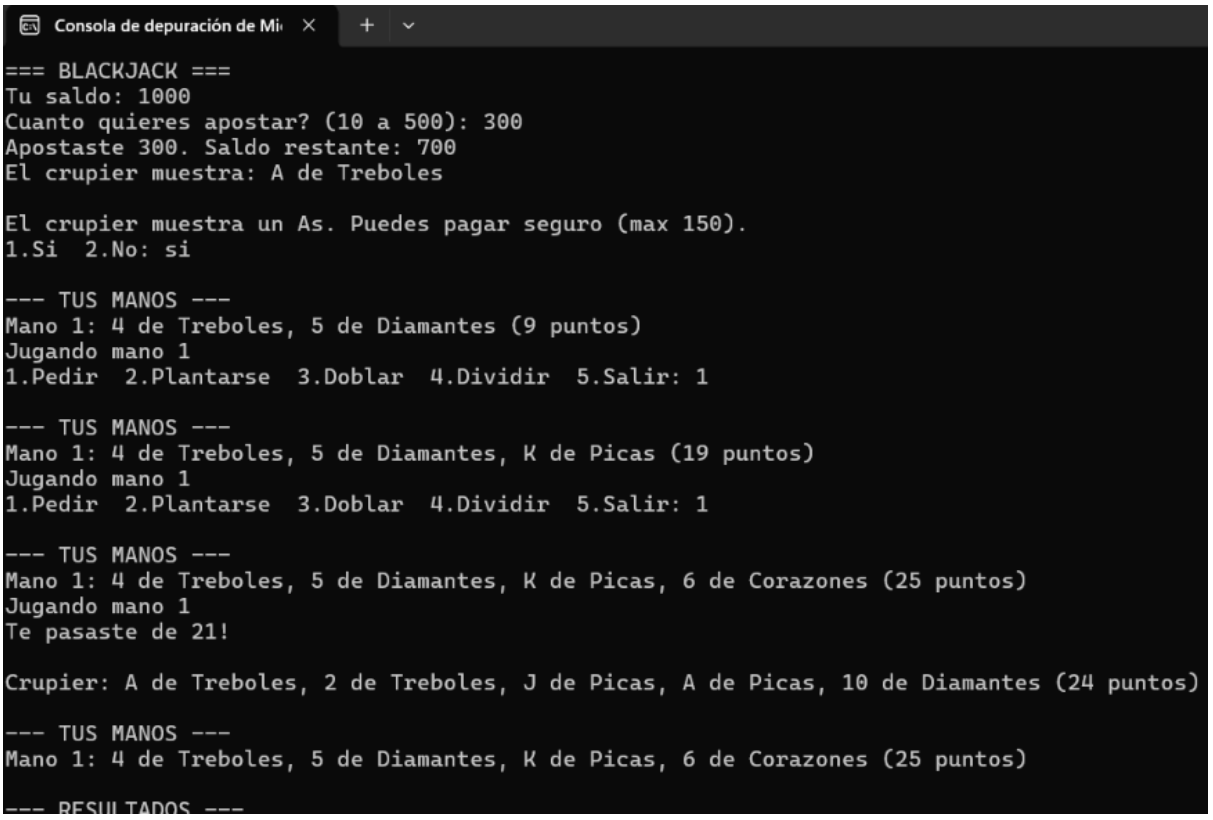
d) Dividir:



e) Doblar:



f) Seguro



5. Conclusión

Con este proyecto logramos crear el juego de Blackjack en consola con C++. El programa trabaja con un zapato de 312 cartas, que son seis mazos completos. Cada jugador inicia con un saldo de ₡ 1000 y puede apostar entre 10 y 500 esto porque se puede jugar con ambas manos. Durante la partida se aplican todas las reglas: se reparten las cartas, se calcula la mano tomando el As como 11 o como 1 según convenga, el jugador puede pedir, plantarse, doblar o dividir, y el crupier pide cartas hasta llegar a 17.

Para organizar el código usamos cinco clases: Carta, Mazo, Mano, Jugador y Crupier. Esto nos ayudó a que el programa fuera más ordenado y fácil de entender, y a que coincidiera con el diagrama UML que presentamos. Además, se validaron las apuestas y las opciones del menú para que el juego no se dañara si el usuario escribía algo incorrecto.

Probamos el juego varias veces y comprobamos los tres resultados posibles: cuando el jugador gana, cuando pierde y cuando empata es muy poca la posibilidad, sin embargo en el código si esta, pero jugando es muy poco probable que este. En cada caso la apuesta se pagó o se devolvió de forma correcta.

Como equipo, aprendimos a dividirnos el trabajo, a usar GitHub para subir nuestros avances y a documentar lo que hacíamos en cada reunión. También practicamos la programación orientada a objetos y el trabajo en equipo. En el futuro, el juego se podría mejorar con una interfaz gráfica o con la opción de jugar varias personas al mismo tiempo.

c) Empate:

En esta partida el jugador y el crupier obtienen el mismo puntaje, por lo que se considera empate y se devuelve la apuesta completa al jugador. No se gana ni se pierde.

Ejemplo: Jugador 18 puntos - Crupier 18 puntos. Resultado: Empate. Se devuelve la apuesta.

[Captura de empate: agrega aqui la captura de una partida con el mismo puntaje para jugador y crupier, donde el programa muestre 'Empate. Se devuelve tu apuesta.']

Para obtener esta captura en la demo, juega hasta que tu puntaje y el del crupier queden iguales, por ejemplo ambos en 18 o ambos en 20.

Conclusion

Con este proyecto logramos crear el juego de Blackjack en consola con C++. El programa trabaja con un zapato de 312 cartas, que son seis mazos completos. Cada jugador inicia con un saldo de 1000 y puede apostar entre 10 y 500. Durante la partida se aplican todas las reglas: se reparten las cartas, se calcula la mano tomando el As como 11 o como 1 segun convenga, el jugador puede pedir, plantarse, doblar o dividir, y el crupier pide cartas hasta llegar a 17 o mas. Tambien se incluyo el seguro cuando el crupier muestra un As.

Para organizar el codigo usamos seis clases: Carta, Mazo, Mano, Jugador, Crupier y Juego. Esto nos ayudo a que el programa fuera mas ordenado y facil de entender, y a que coincidiera con el diagrama UML que presentamos. Ademas, se validaron las apuestas y las opciones del menu para que el juego no se danara si el usuario escribia algo incorrecto.

Probamos el juego varias veces y comprobamos los tres resultados posibles: cuando el jugador gana, cuando pierde y cuando empata. En cada caso la apuesta se pago o se devolvio de forma correcta.

Como equipo, aprendimos a dividirnos el trabajo, a usar GitHub para subir nuestros avances y a documentar lo que haciamos en cada reunion. Tambien practicamos la programacion orientada a objetos y el trabajo en equipo. En el futuro, el juego se podria mejorar con una interfaz grafica o con la opcion de jugar varias personas al mismo tiempo.